

DarkSage

Software Requirements Specification

Version 0.7.0 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: [LightSwornPVP/DarkSage](#)

Source baseline commit: [8b5cf5bd0341d5d6f70c67302c8bce2069d0929a](#)

Generation date: 2026-07-25T16:51:31-04:00

Document ID: DS-002

Document Control

Document ID	DS-002
Title	Software Requirements Specification
Version	0.7.0
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-23
Last Updated	2026-07-25

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Revision History

Version	Date	Author	Summary
0.7.0	2026-07-25	TheSinnerMan / Keeper	Founder Vision Completion: reconciled confirmation and automation language; added research, journal/review, canonical trade-intelligence, live-chart, education, portfolio, strategy, and Discord requirements.
0.1.0	2026-07-23	TheSinnerMan	Document control scaffold created; no normative content.
0.2.0	2026-07-24	TheSinnerMan	First substantial controlled draft. Translates DS-001 and ADR-001–004 into requirement conventions, cross-cutting DS-PRD requirements, and an index of requirement family volumes under requirements/.
0.3.0	2026-07-24	TheSinnerMan	Repair pass addressing independent audit findings DS-002-H01 through H05: corrected the Sage/Risk Engine boundary wording (DS-RSK-001) to permit read/evidence access while preserving the bypass/override prohibition; reclassified 47 unsupported Committed/MVP requirements to Planned after auditing each against DS-001/ADR traceability (Committed/MVP count: 71 → 38 → 35 after a further dependency-consistency pass); resolved Committed-depends-on-Planned inconsistencies by reclassifying the dependent requirements rather than promoting Planned features; clarified undefined market/asset acceptance boundaries as Owner Decision-pending rather than silently delegated to DS-004; added the DS-EDU (Trading Knowledge & Education) requirement family to close a DS-001 §19.2 coverage gap. No MVP scope was invented; the asset-class/product-boundary decision remains an explicit open Owner Decision (Appendix A).

Version	Date	Author	Summary
0.4.0	2026-07-24	TheSinnerMan	Reconciliation pass. Discovered that this repository already contains a mature, pre-existing engineering specification at the root level (PROJECT_SPEC.md, ARCHITECTURE.md, ROADMAP.md, TRADING_RULES.md, SECURITY_RULES.md, AGENTS.md) which the DS-002-H02 repair had not consulted as an "authoritative controlled source." Re-elevated 9 requirements from Planned to Committed/MVP where ROADMAP.md's explicit Phase 1 scope supports it (DS-MKT-002, DS-DAT-003, DS-CHT-001, DS-CHT-002, DS-SCN-001/002/003, DS-RSK-002, DS-WKS-001), each with a Governing Source citation. Updated Appendix A: the MVP asset-class question is now substantially answered (S&P 500 equities first, per PROJECT_SPEC.md §1/§5) rather than fully open. Recorded, but did not attempt to close in this pass, a materially incomplete requirement-family gap: this document has no coverage for the Signal System, the canonical TradeValidationPipeline, Strategy Performance Intelligence, the Mobile client, or Auto-Trader/Execution architecture — all of which are already specified in the root-level documents and should become dedicated requirement families in a future revision. Committed/MVP count: 35 → 44 of 87.

Version	Date	Author	Summary
0.5.0	2026-07-24	TheSinnerMan	Repair pass addressing independent audit findings DS-002-A01 through A04. A01: reclassified DS-AI-001..007 from Committed/MVP to Planned — Sage itself is ROADMAP.md Phase 6, not Phase 1; the underlying safety/authority constraints remain governed unconditionally by DS-PRD-001/002/003/004/005/006/007/008/009/011 regardless of this family's phase. A02: narrowed DS-DAT-003 to its genuine Phase-1 scope (Candle/Quote/Signal identity only), moving the watchlist/position-continuity behavior to a Future Enhancements note tied to those features' own (Planned) classification, removing the prior hard dependency on Planned functionality. A03: added four new requirement families closing a real completeness gap — DS-SIG (Signal System), DS-EXE (Execution, Auto-Trader & Broker — including the product-level TradeValidationPipeline boundary and a Committed live-trading gate), DS-PERF (Strategy Performance Intelligence), DS-MOB (Mobile Client) — each requirement source-traced to ARCHITECTURE.md/PROJECT_SPEC.md/ROADMAP.md/TRADING_RULES.md/SECURITY_RULES.md and phase-classified accordingly. A04: defined controlled default numeric thresholds for DS-MKT-004 (staleness: 10s real-time / delay+60s delayed / 1hr post-close end-of-day) and DS-NFR-001 (startup: 10s cold / 3s warm, with a defined reference-hardware profile), making both testable now rather than deferred to DS-004. Also added cross-cutting DS-PRD-011 (AI Output Validation, Committed) closing the DS-003-A03 gap at the product level.
0.6.0	2026-07-24	TheSinnerMan	Repair pass addressing independent audit finding DS-002-RA01: the v0.5.0 A01 repair correctly reclassified DS-AI-001..007 to Planned but left three Committed requirements (DS-RSK-001, DS-OPS-001, DS-OPS-004) with unqualified dependencies on the newly-Planned Sage requirements. Narrowed all three to their phase-appropriate, Sage-independent obligations, moving Sage-specific query/logging/degradation behavior to Future Enhancements notes. DS-AI family remains Planned.

Table of Contents

Document Control	2
Revision History	3
Table of Contents	6
1. Purpose	8
2. Scope	8
3. Audience	8
4. Definitions	8
5. Requirement Conventions and Classification	9
5.1 Requirement ID Scheme	9
5.2 Normative Language	9
5.3 Release Classification	9
5.4 Priority	10
5.5 Requirement Fields	10
6. Cross-Cutting Product Requirements (DS-PRD)	10
DS-PRD-001 — Model Independence	10
DS-PRD-002 — Evidence Provenance	11
DS-PRD-003 — Presentation Independence	11
DS-PRD-004 — Deterministic Financial Truth	12
DS-PRD-005 — User Decision Authority	13
DS-PRD-006 — Risk Engine Authority	13
DS-PRD-007 — No Unapproved Autonomous Trading	14
DS-PRD-008 — Data State Visibility (Freshness, Delay, History, Simulation)	15
DS-PRD-009 — Uncertainty Communication	15
DS-PRD-010 — Application Startup and Session Initialization	16
DS-PRD-011 — AI Output Validation	17
7. Requirement Family Index	17
8. Target Users and Personas	20
9. Non-Goals and Boundaries	20
10. Dependencies	20
11. Risks and Constraints	21
12. Verification Approach	21
13. References	21

Appendix A — Open Questions

21

1. Purpose

DS-002 is the authoritative product and software requirements specification for DarkSage. It translates the philosophy, principles, and boundaries established in DS-001 — Executive Vision & Product Foundation into requirements that are unambiguous, testable, traceable, and consistent with the approved Architecture Decision Records.

DS-002 does not restate DS-001's philosophy. Where a requirement depends on a DS-001 principle, this document references it rather than re-deriving it.

2. Scope

This document governs:

- the requirement conventions used across the DarkSage Codex's Volume II requirement family documents (ID scheme, normative language, release classification, priority);
- cross-cutting product requirements that apply across multiple functional families (DS-PRD);
- the index of requirement family documents held under docs/codex/Volume-02-Product/requirements/;
- target user/persona classification at the requirements level;
- product-level non-goals and boundaries; and
- open product-requirements questions that remain genuinely unresolved.

DS-002 does not govern detailed Sage reasoning/evidence/memory behavior (DS-003), technical architecture (DS-004), database design (DS-005), API contracts (DS-006), UI/UX system detail (DS-007), security architecture (DS-008), testing/QA procedure (DS-009), development standards (DS-010), roadmap sequencing (DS-011), or individual ADRs (DS-012). DS-002 requirements may reference those volumes but do not substitute for them.

3. Audience

Product owners, engineering contributors, independent auditors, and future Codex authors who need an authoritative, testable statement of what DarkSage shall, should, or may do at the product/software level.

4. Definitions

See DS-001 §24 (Glossary) for foundational terms (DarkSage, Sage, User Authority, Deterministic Financial Truth, Presentation Independence, Workspace Studio, Trading Knowledge Engine). Additional terms used specifically in DS-002:

Term	Meaning
Requirement family	A grouped set of related requirements sharing an ID prefix (e.g., DS-MKT), held in its own file under requirements/

Term	Meaning
Cross-cutting requirement	A requirement whose obligation applies across more than one requirement family; recorded under the DS-PRD prefix in this document
Committed / MVP	A requirement approved for the current minimum viable product scope
Planned	A requirement approved in direction but not committed to the current MVP scope or sequencing
Future / Exploratory	A non-binding requirement expressing direction only; requires its own future approval before becoming Committed or Planned
Material	Financially, analytically, or safety significant enough that an omission or error could mislead a user's decision (used consistently with DS-001 §8.4, §13)

5. Requirement Conventions and Classification

5.1 Requirement ID Scheme

Requirement IDs follow DS-**<DOMAIN>**-NNN per docs/standards/NAMING_AND_ID_STANDARD.md, e.g. DS-RSK-001. NNN is a zero-padded, monotonically increasing integer within its domain family, assigned in authoring order and never reused after removal (a removed requirement's ID is marked Withdrawn, not recycled).

5.2 Normative Language

Per docs/standards/DOCUMENTATION_STANDARD.md §6: **shall** denotes a mandatory requirement; **should** denotes a strong recommendation with permitted, justified exceptions; **may** denotes a permitted option. Every requirement's Description uses exactly one of these terms for its core obligation.

5.3 Release Classification

Every requirement carries exactly one Release Classification:

- **Committed / MVP** — approved for the current minimum viable product. Implementation-ready once dependent architecture exists.
- **Planned** — approved direction, not yet committed to current MVP scope or sequencing. Becomes Committed only through an explicit roadmap decision (DS-011) or owner/Keeper approval, not by silent reinterpretation.
- **Future / Exploratory** — non-binding aspiration consistent with DS-001 §22. Requires future dedicated requirements, risk review, and architecture approval before it may become Planned or Committed.

Committed/MVP eligibility test (established in the DS-002-H02 repair pass): a requirement may be classified Committed/MVP only if it is traceable to (a) a DS-001 section explicitly marked

FOUNDATIONAL PRINCIPLE, (b) an approved ADR (ADR-001–004), or (c) bare technical or safety necessity without which an already-Committed item would be meaningless (e.g., the application must be able to start at all). Direction-only language in DS-001 (sections marked CURRENT PRODUCT DIRECTION, or plain "may"/"should"/"over time" phrasing) does not by itself support a Committed/MVP classification — such items default to Planned until an explicit owner MVP-scope decision promotes them. This is a conservative, non-inventive default: it removes unsupported commitment claims without deciding the MVP on the owner's behalf.

5.4 Priority

Priority (Critical / High / Medium / Low) indicates severity of harm or value loss if unmet, independent of Release Classification. A Committed/MVP requirement is typically Critical or High; a Planned or Future/Exploratory requirement may carry any priority to indicate its eventual importance once committed.

5.5 Requirement Fields

Each material requirement in a family document states: ID, Title, Priority, Release Classification, Status, Purpose, Description, Dependencies, Acceptance Criteria, Edge Cases, Implementation Notes, Testing, and (where applicable) Future Enhancements. Fields follow `docs/templates/markdown/Requirement.md`; DS-002 extends that template with Purpose, Release Classification, Implementation Notes, and Future Enhancements to satisfy this authoring pass's quality bar. Full five-stage traceability (Requirement → Design/ADR → Source → Test → Release/Change) is recorded in `docs/traceability/TRACEABILITY_MATRIX.csv` as Source/Test/Release links become available; at this drafting stage, Dependencies records the Design/ADR link and Source/Test/Release remain Pending.

6. Cross-Cutting Product Requirements (DS-PRD)

The following requirements apply across multiple requirement families and are recorded once here rather than duplicated per family. Family documents reference these by ID instead of restating them.

DS-PRD-001 — Model Independence

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Prevent DarkSage's core product capability from becoming architecturally dependent on a single AI/model vendor or model family, preserving the platform's ability to change providers, add local models, or degrade gracefully.

Description: DarkSage core product capability shall not depend architecturally on one AI/model vendor or model family unless explicitly approved by an ADR. Sage-facing functionality shall be defined in terms of capability contracts (reasoning, synthesis, explanation) rather than a specific vendor API.

Dependencies: DS-001 §12, §21; DS-AI (Sage requirement family)

Acceptance Criteria:

- No Committed/MVP requirement names a specific external AI vendor as a mandatory, non-substitutable dependency.
- Where a provider is used in the current implementation, an abstraction boundary is documented in DS-004/DS-AI such that a substitute provider could be integrated without rewriting product-level requirements.
- Any exception (a requirement that does mandate a specific vendor) is recorded through an ADR, not silently assumed.

Edge Cases:

- A capability that is genuinely only available from one vendor today (e.g., a novel model feature) is recorded as a time-bound implementation note, not a permanent architectural dependency.
- Local/offline model operation, if unavailable at a given time, degrades per DS-AI failure-behavior requirements rather than blocking the product.

Implementation Notes: Detailed provider-abstraction architecture belongs to DS-004; this requirement constrains product-level requirements-writing, not implementation detail.

Testing: Requirements review — reject any new Committed/MVP requirement that hard-codes a vendor without an accompanying ADR.

DS-PRD-002 — Evidence Provenance

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Allow users and auditors to trace a material Sage conclusion back to what produced it.

Description: Material Sage conclusions shall, where technically feasible, be traceable to the contributing evidence, calculations, model outputs, and timestamps that produced them.

Dependencies: DS-001 §8.2, §15; DS-AI; DS-MKT; DS-RSK

Acceptance Criteria:

- A material conclusion (one that could materially affect a user's decision) exposes, on request, its contributing evidence items and their timestamps.
- Deterministic calculation outputs referenced by a conclusion identify the calculation and its inputs, not only the result.
- Where provenance cannot be captured (e.g., a third-party black-box signal), the conclusion discloses that limitation rather than presenting the evidence as fully traceable.

Edge Cases:

- Evidence that has since changed or expired is shown with its original timestamp, not silently updated to appear current.
- Conflicting evidence sources are both disclosed rather than one being silently discarded.

Implementation Notes: "Where technically feasible" acknowledges that some third-party inputs may arrive without full provenance; DS-AI defines minimum disclosure obligations for such cases.

Testing: Evidence-trace inspection test per material Sage conclusion type; verify timestamp and source fields are present and accurate.

DS-PRD-003 — Presentation Independence

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Guarantee that workspace customization never silently changes what the system is capable of, per ADR-004.

Description: Hiding, moving, removing, or rearranging UI widgets shall not silently disable enabled system capability or Sage's access to enabled evidence.

Dependencies: DS-001 §16; ADR-004; DS-WKS; DS-AI

Acceptance Criteria:

- Removing a widget from a workspace layout does not change the set of data sources, calculations, or evidence available to Sage or to other enabled widgets.
- Capability state (what is enabled) and presentation state (what is visible) are stored and evaluated independently.
- A capability change (enable/disable) always requires an explicit, attributable user or administrative action distinct from a layout edit.

Edge Cases:

- A widget that is the *only* current UI surface for a capability may still be hidden; the capability itself (e.g., underlying data feed) remains enabled and available to Sage even with no visible surface.
- Restoring a previously hidden widget shall not require re-enabling any capability — the capability was never disabled.

Implementation Notes: Enforced architecturally per DS-004; DS-002 states the product-level obligation and its testability.

Testing: Regression test: hide/remove each MVP widget in turn and confirm no change in Sage evidence availability or calculation results.

DS-PRD-004 — Deterministic Financial Truth

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Ensure users never receive a materially wrong number because a generative model, rather than a deterministic implementation, produced it.

Description: Material financial calculations shall use deterministic, testable implementations as the authoritative source; generative AI output shall not be used as the authoritative source for a material financial calculation.

Dependencies: DS-001 §9, §11 (Constitution #6), §21; ADR-003; DS-RSK; DS-BKT; DS-PRT

Acceptance Criteria:

- Every material financial figure presented to a user (position value, P/L, risk metric, backtest statistic) traces to a deterministic calculation implementation, not to a language-model completion.
- Given identical inputs, a material financial calculation produces identical output across repeated runs (determinism is verifiable by test).
- Sage may explain, summarize, or contextualize a deterministic result; Sage's explanation is never substituted as the number itself.

Edge Cases:

- A calculation with an inherently probabilistic model component (e.g., a Monte Carlo scenario) discloses its stochastic nature and seed/methodology; it is not presented as a single

deterministic fact unless the presented figure (e.g., a percentile) is itself deterministically reproducible from stored inputs.

- If a deterministic calculation is unavailable (e.g., missing data), the system reports the gap rather than allowing a generative estimate to fill it silently.

Implementation Notes: DS-RSK defines the Risk Engine's authority over risk-calculation determinism; DS-BKT defines backtest determinism; DS-004 defines the calculation-engine architecture.

Testing: Deterministic-output regression suite: fixed input fixtures must reproduce identical output byte-for-byte (or within defined floating-point tolerance) across runs and across code versions unless the calculation itself changed (with a recorded change entry).

DS-PRD-005 — User Decision Authority

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Preserve the user as the final decision-maker for consequential actions.

Description: Sage may recommend, explain, compare, prepare, and warn, but the user shall retain authority over consequential investment, trading, and automation actions. User authority may be exercised either through a distinct per-action confirmation or through explicit prior activation of a bounded automation policy with defined permissions, limits, and emergency controls.

Dependencies: DS-001 §8.3, §11 (Constitution #1, #7), §12; DS-AI; DS-RSK

Acceptance Criteria:

- Sage never directly executes a consequential action. A consequential action proceeds only through either (a) a distinct, attributable user confirmation or (b) a previously confirmed, bounded automation policy whose permissions, limits, and current state are visible and auditable.
- Recommendation language is distinguishable from confirmed-action language in the UI and in Sage's own output (e.g., "I recommend..." vs. an executed state).
- Users can decline, question, or ignore a Sage recommendation without being blocked from proceeding with their own decision, subject only to Risk Engine controls (DS-PRD-006).

Edge Cases:

- A user who repeatedly overrides Sage's warnings is not silently rate-limited or degraded in future capability as a consequence; friction, if any, must be explicit and disclosed.
- Bulk or automated user-configured actions (e.g., a saved rule the user explicitly enabled) are distinguished from unrequested autonomous Sage action.

Implementation Notes: Detailed Sage behavioral boundaries belong to DS-003.

Testing: UI/behavioral tests confirm that consequential actions require either an attributable per-action confirmation or an active, previously confirmed automation policy; Sage never serves as the confirming or executing authority.

DS-PRD-006 — Risk Engine Authority

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Guarantee that risk controls remain enforceable independent of Sage, per ADR-002.

Description: Sage shall not bypass or silently override Guardian / Risk Engine rules. Risk Engine determinations shall remain independently enforceable regardless of Sage's output.

Dependencies: DS-001 §13; ADR-002; DS-RSK; DS-AI

Acceptance Criteria:

- A Risk Engine block or limit cannot be circumvented by a Sage-issued instruction, prompt, or generated output.
- Sage's own output surfaces a Risk Engine block/limit when relevant rather than concealing or reinterpreting it.
- Any change to a Risk Engine rule requires an explicit, attributable user or administrative action through a defined risk-configuration surface, not a Sage side effect.

Edge Cases:

- If Sage and the Risk Engine disagree (e.g., Sage suggests an action the Risk Engine would block), the disagreement is surfaced to the user rather than silently resolved in Sage's favor.
- Risk Engine unavailability (e.g., service degradation) shall not be treated as an implicit "allow" — see DS-RSK for fail-safe behavior.

Implementation Notes: DS-RSK defines the Risk Engine's calculation and enforcement requirements; DS-004/DS-008 define architectural/security enforcement of this boundary.

Testing: Adversarial test: attempt to have Sage instruct or narrate around a known Risk Engine limit; confirm the limit still applies and the attempt is logged.

DS-PRD-007 — No Unapproved Autonomous Trading

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Prevent unapproved execution authority while preserving Founder-approved bounded automation modes.

Description: DarkSage shall not permit unapproved, self-escalating, or Sage-directed autonomous execution. Unattended execution is permitted only within a separately defined automation mode that the user explicitly activates and that remains bounded by deterministic strategy logic, permissions, exposure limits, buying-power checks, market-condition checks, execution validation, broker controls, auditability, and emergency-stop behavior.

Dependencies: DS-001 §13, §26.5; DS-RSK; DS-STR; DS-EXE; ADR-002; ADR-005

Acceptance Criteria:

- Advisory Only and Confirmation Required modes never place an order without the required user action.
- Full-Auto Paper may operate unattended only after explicit activation and remains clearly simulated.
- Restricted Full-Auto Live remains unavailable until its dedicated release gates, independent security review, broker permissions, risk controls, reconciliation, and operational readiness requirements are satisfied.
- Sage cannot directly submit, approve, modify, cancel, or route an order; approved deterministic services perform those actions through the canonical validation pipeline.
- The active automation mode, scope, limits, and emergency controls are always visible and auditable.

Edge Cases:

- Disabling, expiring, or invalidating an automation policy stops new order creation fail-closed.
- An alert, Discord message, Sage conversation, or external-channel reaction is notification-only unless a future remote-control design is separately approved with strong authentication and explicit authority.

Implementation Notes: DS-EXE owns detailed automation-mode behavior. ADR-005 records the durable five-mode decision.

Testing: Mode-matrix tests verify allowed and forbidden actions for every automation state, including restart, stale data, provider outage, policy expiry, and emergency-stop transitions.

DS-PRD-008 — Data State Visibility (Freshness, Delay, History, Simulation)

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Prevent a user from mistaking stale, delayed, historical, or simulated data for current live data.

Description: Users and Sage shall be able to distinguish stale, delayed, historical, simulated, and current data wherever that distinction is material to a decision.

Dependencies: DS-001 §19; DS-MKT; DS-BKT; DS-AI

Acceptance Criteria:

- Every material data value displayed carries a discoverable state indicator (current / delayed / stale / historical / simulated) and, where applicable, a timestamp.
- Sage's narrative output referencing a data value states its state/timestamp when the state is anything other than current.
- Backtest and simulated output is visually and textually distinguishable from live data in every surface it appears.

Edge Cases:

- A data feed that silently stops updating is surfaced as stale after a defined staleness threshold (see DS-MKT), not left showing a last-known value indistinguishable from current.
- Mixed-state views (some symbols current, some delayed) label each item individually rather than applying one blanket label to the view.

Implementation Notes: Staleness thresholds and per-feed definitions belong to DS-MKT.

Testing: Feed-interruption simulation test verifying stale/delayed indicators appear within the defined threshold window.

DS-PRD-009 — Uncertainty Communication

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Prevent probabilistic output from being mistaken for guaranteed fact.

Description: Probabilistic or model-generated forecasts shall not be represented as deterministic future facts.

Dependencies: DS-001 §8.4, §13, §15; DS-AI; DS-RSK; DS-BKT

Acceptance Criteria:

- Any forward-looking, probability-based, or model-generated statement (forecast, scenario probability, scored ranking) is labeled as such in the surface where it appears, not presented with the same visual/textual authority as an observed historical fact.
- Confidence or uncertainty is communicated using consistent, defined language or scale, not ad hoc wording that varies by feature.
- Backtest results are never presented as a guarantee of future performance (explicit disclosure required per DS-BKT).

Edge Cases:

- A high-confidence model output is still labeled as a model output; confidence level does not upgrade a forecast to a "fact" presentation.
- Absence of sufficient evidence is communicated as "uncertain / insufficient evidence," not silently omitted or defaulted to a neutral-looking value.

Implementation Notes: A shared confidence/uncertainty vocabulary is defined once (DS-AI) and reused across families rather than redefined per feature.

Testing: Content-review test across all forecast/score-producing surfaces confirming presence of an uncertainty label and absence of guarantee language.

DS-PRD-010 — Application Startup and Session Initialization

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Ensure the application reaches a usable, trustworthy state predictably on launch.

Description: DarkSage shall initialize to a usable workspace state on startup, communicating the status of required local and external dependencies (data feeds, local storage, configured integrations) before presenting data as current.

Dependencies: DS-USR (onboarding); DS-WKS (saved layout restore); DS-MKT (feed status); DS-OPS (startup logging)

Acceptance Criteria:

- On a normal launch, the application reaches a usable state within a startup budget defined in DS-NFR-001 — restoring prior workspace state where layout persistence exists (DS-WKS-003, Planned pending that feature's implementation), or presenting a default view otherwise.
- If a required dependency (e.g., local data store) fails to initialize, the user is shown an understandable error rather than a silently degraded or misleading UI.
- Data displayed immediately after startup is marked with its actual state (see DS-PRD-008) until freshness is confirmed.

Edge Cases:

- First-run startup with no saved layout or prior configuration routes to onboarding (DS-USR) rather than an empty/broken workspace.
- Startup while offline succeeds in a degraded mode per DS-OPS rather than blocking entirely, where local-only capability remains usable.

Implementation Notes: Startup performance targets are defined once in DS-NFR and referenced here rather than duplicated.

Testing: Cold-start test (clean profile), warm-start test (existing profile), and offline-start test, each verifying correct state communication.

DS-PRD-011 — AI Output Validation

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Close a gap identified in the DS-003-A03 audit finding: `SECURITY_RULES.md` treats all AI output as untrusted input requiring validation, but no DS-002 cross-cutting requirement stated this. This is a safety guardrail that applies the moment any AI feature exists, regardless of that feature's own phase/classification — the same reasoning as DS-PRD-006/007.

Description: Structured output produced by any AI provider (local or cloud, any vendor) shall be validated before any application use or state transition; AI output shall never directly execute shell commands, modify security settings, submit broker orders, change live credentials, or override risk controls. Validation shall fail closed (reject/ignore the output) when validation cannot be completed with confidence.

Dependencies: `SECURITY_RULES.md` "AI Output Is Untrusted Input"; DS-PRD-001, DS-PRD-004, DS-PRD-006, DS-PRD-007

Acceptance Criteria:

- No code path applies raw AI output to a security-sensitive or trading-sensitive state transition without a validation step in between.
- A validation failure results in the output being rejected/ignored (fail closed), not applied with a warning.
- This requirement applies to every AI feature regardless of that feature's own Release Classification (e.g., it already governs the Planned DS-AI/DS-SGE families now, before they are built).

Edge Cases:

- AI output that is itself just conversational text (no state transition) is not blocked by this requirement — it applies to output that would drive an action or a security-relevant state change.

Implementation Notes: DS-004/DS-008 concern for the validation-layer architecture.

Testing: Adversarial test: craft AI output attempting a disallowed action (e.g., embedded shell-command-like text, a fabricated "override risk controls" instruction) and confirm it is rejected before reaching any sensitive code path.

7. Requirement Family Index

Detailed functional requirements are held in per-family documents under `docs/codex/Volume-02-Product/requirements/`. This index is authoritative for which families exist and their scope boundary; the family documents are authoritative for their own requirement content.

Prefix	Family	File	Summary Scope
DS-PRD	Product/Platform Behavior	(this document, §6)	Cross-cutting product mandates; application startup
DS-USR	Users, Personas & Onboarding	requirements/DS-USR-Users-and-Onboarding.md	Onboarding, user preferences, terminology mode, capability-based personas
DS-WKS	Workspace Studio	requirements/DS-WKS-Workspace-Studio.md	Configurable workspaces, drag/drop layout, saved layouts, multi-monitor
DS-MKT	Market Data & Intelligence (Observatory)	requirements/DS-MKT-Market-Data.md	Ingestion, historical data, real-time/delayed handling, calendars, corporate actions, watchlists, symbol identity reference
DS-SCN	Scanner (Watchtower)	requirements/DS-SCN-Scanner.md	Scanning, deterministic pre-filtering, ranking/scoring
DS-SIG	Signal System	requirements/DS-SIG-Signal-System.md	Signal representation, grading, why-trade/why-not-trade, expiration
DS-CHT	Charts	requirements/DS-CHT-Charts.md	Charting and technical analysis surfaces
DS-AI	Sage	requirements/DS-AI-Sage.md	Conversational interaction, evidence access, explainability, confidence, provider abstraction, failure/degradation
DS-RSK	Risk Engine (Guardian)	requirements/DS-RSK-Risk.md	Risk Engine authority, deterministic calculations, position/risk metrics, scenario analysis
DS-STR	Strategies (Forge)	requirements/DS-STR-Strategies.md	Strategy construction and validation
DS-PERF	Strategy Performance Intelligence	requirements/DS-PERF-Strategy-Performance.md	Performance metrics, segmentation, Strategy DNA, anti-overfitting safeguards

Prefix	Family	File	Summary Scope
DS-BKT	Backtesting (Chronicle)	requirements/DS-BKT-Backtesting.md	Historical backtesting, cost/slippage realism, look-ahead bias prevention
DS-PRT	Portfolio (Treasury)	requirements/DS-PRT-Portfolio.md	Portfolio tracking, positions, transactions, realized/unrealized performance
DS-EXE	Execution, Auto-Trader & Broker	requirements/DS-EXE-Execution-and-Broker.md	TradeValidationPipeline product boundary, Auto-Trader states, Emergency Stop/Flatten, broker adapter, live-trading gate
DS-MOB	Mobile Client	requirements/DS-MOB-Mobile-Client.md	Mobile scope, client-only boundary, mobile security
DS-ALT	Alerts & Monitoring	requirements/DS-ALT-Alerts.md	Alerts and notifications
DS-RSH	Research Intelligence	requirements/DS-RSH-Research-Intelligence.md	Evidence-governed news, filings, earnings, macro, insider/political disclosures, catalysts, and thesis monitoring
DS-JRN	Journal & Review Intelligence	requirements/DS-JRN-Journal-and-Review.md	Trade journal, behavioral reflection, daily/weekly review, lessons, and process improvement
DS-DAT	Data Management	requirements/DS-DAT-Data-Management.md	Local data storage, import/export, symbol/security identity
DS-EDU	Trading Knowledge & Education	requirements/DS-EDU-Trading-Knowledge.md	Contextual terminology reference; future Trading Knowledge Engine (DS-001 §19.2)
DS-INT	Integrations	requirements/DS-INT-Integrations.md	External integration boundaries
DS-SEC	Security & Privacy	requirements/DS-SEC-Security-and-Privacy.md	Privacy, credential/secrets handling
DS-NFR	Non-Functional Requirements	requirements/DS-NFR-Non-Functional.md	Accessibility, performance, reliability, maintainability, extensibility, testing expectations

Prefix	Family	File	Summary Scope
DS-OPS	Reliability, Observability & Operations	requirements/DS-OPS-Operations.md	Logging, auditability, error handling, offline/degraded operation

Additional prefixes may be added only when genuinely needed and consistent with docs/standards/NAMING_AND_ID_STANDARD.md; any addition updates this index in the same change.

8. Target Users and Personas

Per DS-001 §18, DarkSage serves broad groups that value explainable decision support. DS-002 defines three capability-based user groups for requirements purposes (detailed in DS-USR):

- **Developing / self-directed investor** — learning markets, values explanation and guardrails over raw power.
- **Active trader** — values speed of access to relevant information, scanning, and alerting.
- **Advanced / professional-style user** — values analytical depth, customization, and minimal friction from simplified defaults.

These are capability-based groups, not demographic assumptions; a single user may move between them. The product shall remain approachable without artificially limiting advanced capability (DS-001 §18).

9. Non-Goals and Boundaries

Consistent with DS-001 §21, DS-002 requirements shall not be interpreted to imply that DarkSage is:

- a promise or guarantee of profit;
- a guaranteed market predictor or oracle;
- a substitute for user judgment;
- an autonomous trading system (see DS-PRD-007);
- dependent on visible widgets for analytical capability (see DS-PRD-003);
- required to use AI for tasks that are properly deterministic (see DS-PRD-004; DS-001 §9);
- committed to every platform, asset class, or integration mentioned as future direction in DS-001 §22.

10. Dependencies

- DS-001 — Executive Vision & Product Foundation (v1.0.0, Approved) — governing philosophy and boundaries.
- ADR-001 — Desktop-First Application
- ADR-002 — Sage Cannot Bypass the Risk Engine
- ADR-003 — Deterministic Financial Calculations
- ADR-004 — Presentation Independence

11. Risks and Constraints

- **Sequencing risk:** DS-002 is authored before DS-004 (Technical Architecture); some Implementation Notes are necessarily provisional and marked as delegated to DS-004 rather than prescribed here.
- **Scope discipline risk:** the breadth of DS-001's product definition (§4) creates pressure to over-commit MVP scope. This draft mitigates that by using Release Classification consistently and defaulting ambiguous items to Planned rather than Committed.
- **Terminology consistency risk:** dual professional/Codex-themed terminology (DS-001 §17) must remain consistent across all family documents; DS-USR owns the terminology-mode requirement and other families reference it rather than redefining it.

12. Verification Approach

Each requirement family document defines feature-level Testing per requirement. At the DS-002 level, verification of this document itself consists of: unique-ID verification across all families, cross-reference consistency against DS-001/ADR-001–004, Release Classification consistency, and Markdown/metadata structural checks, recorded in `HANDOFF.md` for this task.

13. References

- `docs/CODEX_INDEX.md`
- `docs/standards/DOCUMENTATION_STANDARD.md`
- `docs/standards/STYLE_GUIDE.md`
- `docs/standards/WRITING_GUIDE.md`
- `docs/standards/NAMING_AND_ID_STANDARD.md`
- `docs/templates/markdown/Requirement.md`
- `docs/traceability/README.md`
- `docs/codex/Volume-01-Foundation/DS-001-Executive-Vision.md`
- `docs/codex/Volume-12-ADRs/ADR-001-Desktop-First-Application.md`
- `docs/codex/Volume-12-ADRs/ADR-002-Sage-Cannot-Bypass-the-Risk-Engine.md`
- `docs/codex/Volume-12-ADRs/ADR-003-Deterministic-Financial-Calculations.md`
- `docs/codex/Volume-12-ADRs/ADR-004-Presentation-Independence.md`
- `PROJECT_SPEC.md`, `ARCHITECTURE.md`, `ROADMAP.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, `AGENTS.md` (repository-root engineering specification; authoritative controlled sources per the DS-002-H02 Committed/MVP eligibility test, consulted starting with the v0.4.0 revision)

Appendix A — Open Questions

The following are genuine owner-decision items, not routine implementation detail, and are not answered by this draft:

1. **MVP scope and asset-class boundary — substantially answered by a source discovered after the DS-002-H02 repair.** DS-001 itself deliberately does not commit to

specific features, markets, asset classes, or acceptance thresholds (§4). However, PROJECT_SPEC.md §1/§5 and ROADMAP.md's Phase 1 ("Core Market Intelligence") section — pre-existing, authoritative, repository-root engineering documents that predate this Codex volume — already establish: initial asset-class scope is US equities, initially S&P 500-focused; initial feature scope is market data ingestion, the core indicator set, the scanner, dual-chart-engine charting, and a basic desktop shell; options, live trading, and mobile are explicitly later phases (Phase 11, Phase 13–14, Phase 9 respectively). This draft's v0.4.0 revision re-elevated 9 requirements to Committed/MVP accordingly (see Governing Source citations on each), and the v0.5.0 repair defined concrete default staleness/startup thresholds directly in DS-MKT-004/DS-NFR-001 (see their Governing Threshold notes), closing the DS-002-A04 finding. What remains genuinely open is formal owner sign-off that ROADMAP.md's phase boundaries are themselves the intended release-scope authority for Codex purposes (a governance/process confirmation, not a product invention).

2. Live brokerage/execution integration timing — DS-001 §21 and DS-PRD-007 exclude autonomous trading from current scope, but do not state whether *manual*, *user-confirmed* order placement through a connected brokerage is itself Committed/MVP, Planned, or Future/Exploratory for the current release. DS-INT/DS-PRT record this as Planned pending an explicit decision.

3. Multi-monitor support commitment level — DS-001 §19.1 mentions multi-monitor workflows as part of the Workspace Studio vision without committing to it. DS-WKS classifies it Future/Exploratory pending an owner decision on whether it belongs in a nearer-term Planned scope.

1. RESOLVED in v0.5.0. Missing requirement-family coverage identified in the v0.4.0 reconciliation pass (Signal System, TradeValidationPipeline, Strategy Performance Intelligence, Mobile client, Auto-Trader/Execution/Broker architecture) is now addressed by the new DS-SIG, DS-EXE, DS-PERF, and DS-MOB families (DS-002-A03 repair). Retained here for revision-history traceability, not as an open item.

Routine implementation details (specific UI component choices, specific data-vendor selection) are intentionally left to DS-004/DS-005/DS-006/DS-007 and are not recorded as SRS-level open questions.